

HDL Design Homework: SystemC vs SystemVerilog

Yousef Awad
EEL 4783 - HDL in Digital Systems

March 2, 2026

Part 1: SystemC

1.1: Counter Module

a) The completed module skeleton

```
1 #include <systemc>
2
3 using namespace sc_core;
4 using namespace sc_dt;
5
6 SC_MODULE(Counter4) {
7     sc_in<bool> clk, reset;
8     sc_out<sc_uint<4>> count;
9     sc_uint<4> current_val; // Internal state
10
11     void count_proc() {
12         // Synchronous active-high reset
13         if (reset.read() == true) {
14             current_val = 0;
15         } else {
16             // sc_uint<4> automatically wraps at 15
17             current_val = current_val + 1;
18         }
19         count.write(current_val);
20     }
21
22     SC_CTOR(Counter4) {
23         SC_METHOD(count_proc);
24         sensitive << clk.pos(); // Trigger on rising edge of clock
25     }
26 };
```

b) Why is SC_METHOD (not SC_THREAD) appropriate here?

SC_METHOD is ideal because it executes its entire block of code and returns control to the simulator without needing to be suspended. Since the requirements explicitly state not to use `wait()`, SC_METHOD is the correct choice over SC_THREAD (which requires `wait()` to suspend and resume execution).

1.2: Testbench Snippet

```
1 #include "counter.cpp"
2 #include <systemc>
3
4 using namespace sc_core;
5 using namespace sc_dt;
6
7 int sc_main(int argc, char *argv[]) {
8     // Instantiate signals and clock (10ns period, 50% duty cycle)
9     sc_clock clk("clk", 10, SC_NS, 0.5);
10    sc_signal<bool> reset;
11    sc_signal<sc_uint<4>> count;
12
13    // Instantiate Counter4
14    Counter4 dut("dut");
15    dut.clk(clk);
16    dut.reset(reset);
17    dut.count(count);
18
19    // Enable VCD tracing
20    sc_trace_file *wf = sc_create_vcd_trace_file("counter_trace");
21    sc_trace(wf, clk, "clk");
22    sc_trace(wf, reset, "reset");
23    sc_trace(wf, count, "count");
24
25    // Stimulus and Simulation Control
26    std::cout << "Starting simulation..." << std::endl;
27
28    reset.write(true); // Assert reset
29    sc_start(15, SC_NS); // For first 15ns
30
31    reset.write(false); // Release reset
32
33    // Run for the remainder of the 200ns total simulation time
34    sc_start(185, SC_NS);
35
36    // Clean up
37    std::cout << "Simulation finished. Trace saved to counter_trace.vcd"
38              << std::endl;
39    sc_close_vcd_trace_file(wf);
40
41    return 0;
42 }
```

Part 2: SystemVerilog

2.1: Shift Register RTL

```
1 `timescale 1ns / 10ps
2
3 module shift_reg #(
4     parameter DATA_W = 8
5 ) (
6     input logic clk,
7     reset,
8     shift_en,
9     input logic [DATA_W-1:0] data_in,
10    output logic [DATA_W-1:0] data_out
11 );
12
13 // Array of registers for internal stages as hinted
14 logic [DATA_W-1:0] stages[3:0];
15
16 always_ff @(posedge clk) begin
17     if (reset) begin
18         stages[0] <= '0;
19         stages[1] <= '0;
20         stages[2] <= '0;
21         stages[3] <= '0;
22     end else if (shift_en) begin
23         stages[0] <= data_in;
24         stages[1] <= stages[0];
25         stages[2] <= stages[1];
26         stages[3] <= stages[2];
27     end
28     // If shift_en == 0, state is naturally held in always_ff
29 end
30
31 // Continuous assignment to the output port
32 assign data_out = stages[3];
33
34 endmodule
```

2.2: Testbench Logic

```
1 `timescale 1ns / 10ps
2
3 module shift_reg_tb;
4
5     // Inputs to the DUT are declared as 'reg'
6     reg clk;
7     reg reset;
8     reg shift_en;
9     reg [7:0] data_in;
10
11     // Outputs from the DUT are declared as 'wire'
12     wire [7:0] data_out; // Assumed 8-bit output, adjust if your module differs
13
14     // match your actual Verilog design file.
15     shift_reg dut (
16         .clk(clk),
17         .reset(reset),
18         .shift_en(shift_en),
```

```

19     .data_in(data_in),
20     .data_out(data_out)
21 );
22
23 // Generates a clock with a 10ns period, matching the #10 step delays in your
    stimulus
24 initial begin
25     clk = 0;
26     forever #5 clk = ~clk;
27 end
28
29 // Your Stimulus Block
30 initial begin
31     // Dumpfile setup for "shift.vcd"
32     $dumpfile("shift.vcd");
33     // Adjusted to match the testbench module name 'shift_reg_tb'
34     $dumpvars(0, shift_reg_tb);
35
36     // Reset sequence
37     reset = 1;
38     shift_en = 0;
39     data_in = 8'hAA;
40     #15;
41     reset = 0;
42
43     // Cycles 1-4: shift_en = 1, incrementing data_in
44     shift_en = 1;
45     data_in = 8'h11;
46     #10;
47     data_in = 8'h22;
48     #10;
49     data_in = 8'h33;
50     #10;
51     data_in = 8'h44;
52     #10;
53
54     // Cycles 5-6: shift_en = 0 (hold state)
55     shift_en = 0;
56     #20;
57
58     // Cycle 7: shift_en = 1, data_in = 8'hFF
59     shift_en = 1;
60     data_in = 8'hFF;
61     #10;
62
63     // Wait remaining time to hit 100ns total simulation time
64     // Time elapsed: 15 (reset) + 40 (c1-4) + 20 (c5-6) + 10 (c7) = 85ns
65     #15;
66
67     $finish;
68 end
69
70 endmodule

```

2.3: Concept Check

a) Why does SystemC use `sc_signal` instead of plain C++ variables for ports?

`sc_signal` implements the evaluate-update paradigm of hardware simulation, ensuring that updates are deferred until the next simulation delta cycle to prevent race conditions that plain C++ variables would cause in concurrent processes.

b) What critical error occurs if you declare `data_out` as `wire` instead of `logic` in Problem 2.1?

If you attempt to assign to `data_out` procedurally (such as inside an `always_ff` block) while it is declared as a `wire`, the SystemVerilog compiler will throw an error because `wire` types can only be driven by continuous assignments (`assign`), whereas `logic` can be driven procedurally.